

Color space



Contents

1 색상의 개념

1.1 RGB시스템

1.2 HSL - 색조 채도 밝기

1.3 HSV - 색조 채도 명도

2 addWeighted

2.1 뭐 import하고

2.2 평화로운 일상

2.3 요쁜 이미지 하나

2.4 두 이미지는 크기가 다르군요

2.5 잠깐~ addWeighted 함수의 옵션의 의미

2.6 그래서 addWeighted를 시켰더니.. 에러가~

2.7 resize 시키고~

2.8 합치기~

2.9 다시 resize

2.10 이건 offset 작업

2.11 픽셀 교체~

2.12 흠...

3 워터마크

3.1 다시 시작해봅시다

3.2 offset을 계산하고

3.3 offset에 의해 잘린 영역만 본다면~

3.4 잠깐. roi란?

3.5 일단 img2를 gray로

3.6 gray에서는 255가 흰색, 0이 검은색~

3.7 이 관계를 뒤집는다

3.8 gray니까 shape은 2차원

3.9 잠깐 bitwise_and

3.10 bitwise_or

3.11 bitwise_xor

3.12 까망색배경, 즉, 0으로 배경이 채워지게~

3.13 roi에다가 덮어버림

3.14 짤!

4 mask 연습

4.1 이런 소스에

4.2 mask가 될 그림을 가져오고

4.3 이 배경에 넣고 싶다면~

4.4 이번에는 copyTo

4.5 이렇게 하고 싶다는 거지

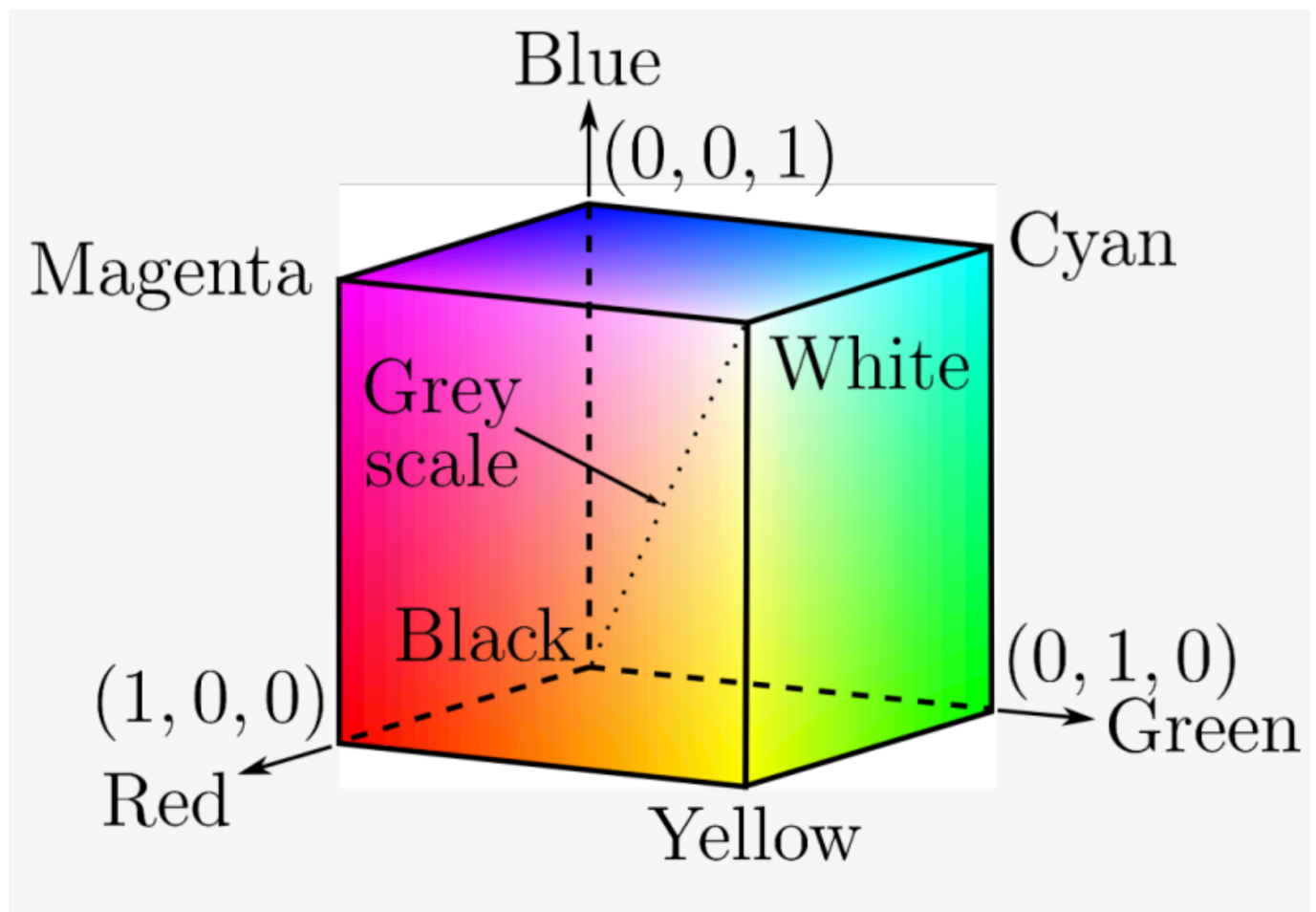
4.6 황당할 만큼 간단함

Color space

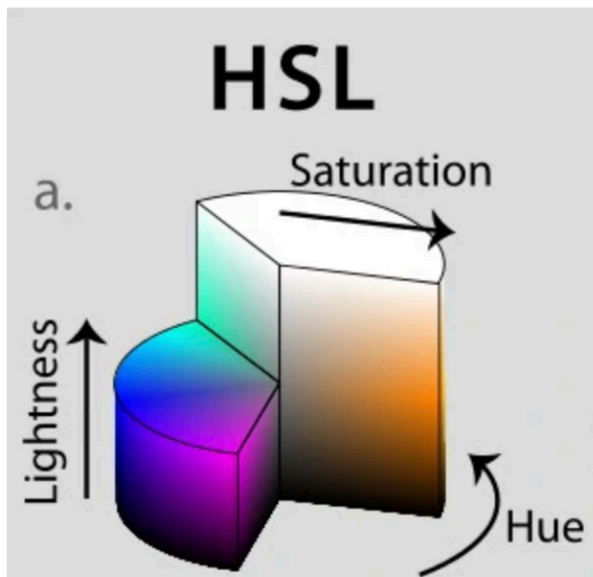
Exported from Confluence · <https://pinkwink.atlassian.net>

색상의 개념

RGB시스템

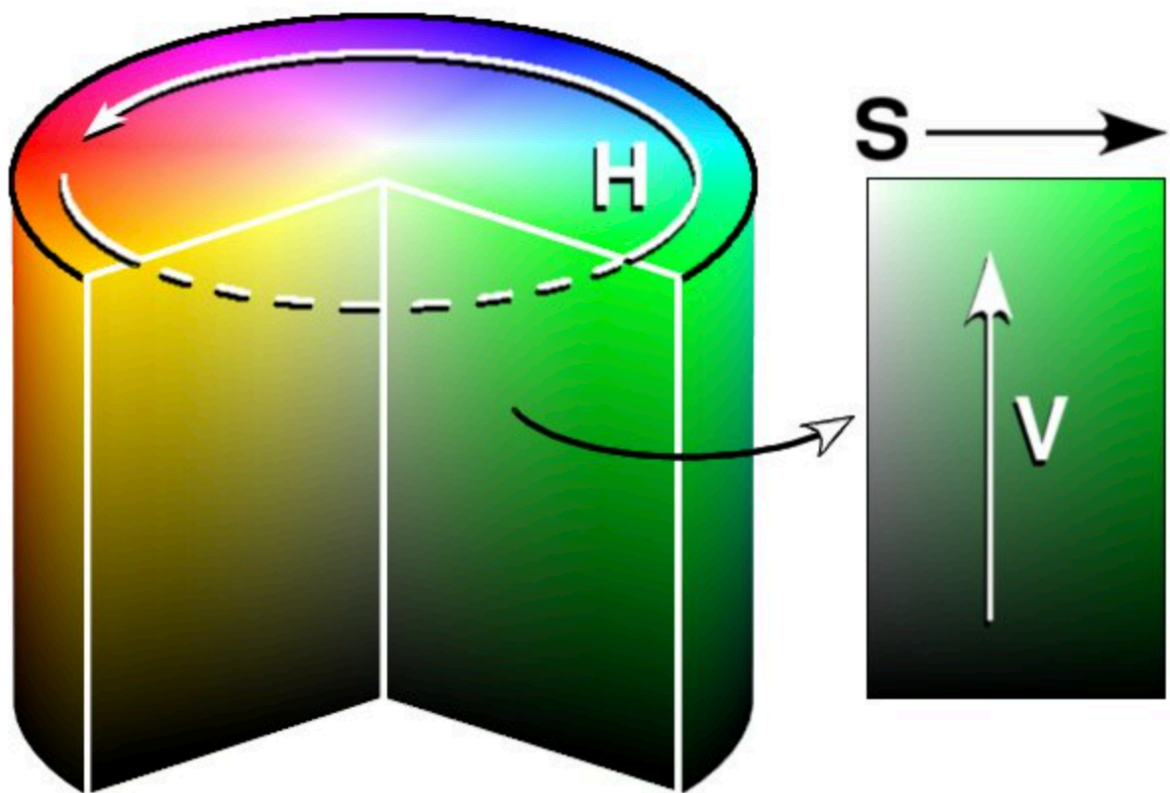


HSL - 색조 채도 밝기



- 채도 : 색의 진함과 옅음
- 밝기 : 색의 밝음 정도

HSV - 색조 채도 명도



addWeighted

뭐 import하고

```
import cv2
import matplotlib.pyplot as plt
```

Python

평화로운 일상

```
img1 = cv2.imread("./data/sample.jpg")
img1 = cv2.cvtColor(img1, cv2.COLOR_BGR2RGB)
plt.imshow(img1)
```

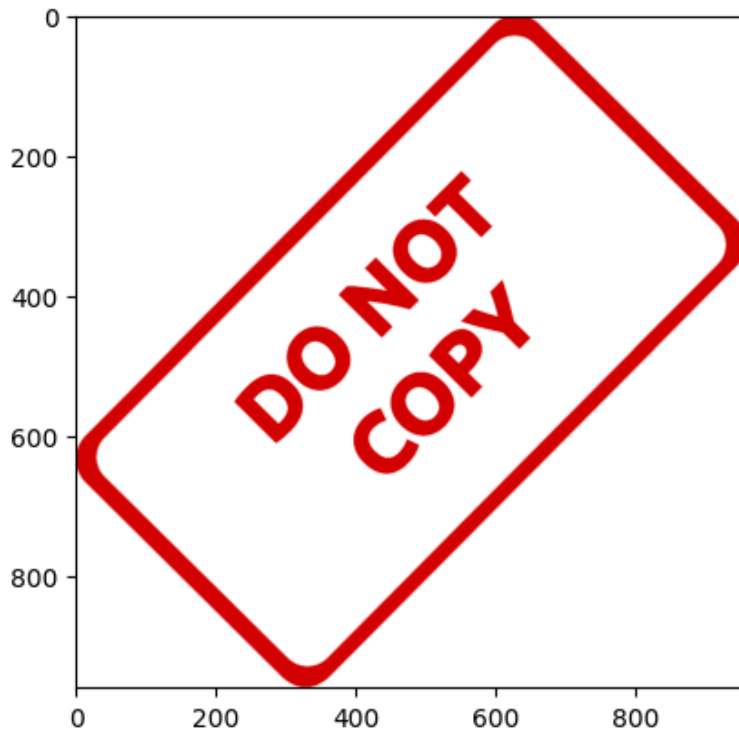
Python



요단 이미지 하나

```
img2 = cv2.imread("./data/watermark_no_copy.png")
img2 = cv2.cvtColor(img2, cv2.COLOR_BGR2RGB)
plt.imshow(img2)
```

Python



두 이미지는 크기가 다르군요

```
img1.shape, img2.shape
```

Python

```
((1000, 1500, 3), (961, 957, 3))
```

잠깐~ **addWeighted** 함수의 옵션의 의미

$$dst = \alpha * src1 + \beta * src2 + \gamma$$

$$\alpha + \beta = 1, \quad 0 \leq \alpha, \beta \leq 1$$

그래서 **addWeighted**를 시켰더니.. 에러가~

```
blended = cv2.addWeighted(src1=img1, alpha=0.5, src2=img2, beta=0.5, gamma=0)
plt.imshow(blended)
```

Python

```
-----
error                                Traceback (most recent call last)
Cell In[59], line 1
----> 1 blended = cv2.addWeighted(src1 = img1, alpha=0.5, src2 = img2, beta = 0.5,
      gamma = 0)
      2 plt.imshow(blended);

error: OpenCV(4.11.0) /io/opencv/modules/core/src/arithm.cpp:662: error: (-209:Sizes
of input arguments do not match) The operation is neither 'array op array' (where
arrays have the same size and the same number of channels), nor 'array op scalar', nor
'scalar op array' in function 'arithm_op'
```

- 에러가 발생 했을경우 겁먹지 말고 에러 메시지를 끝에서부터 잘 살펴 보세요. 중요한 힌트가 있습니다.

resize 시키고~

```
img2 = cv2.resize(img2, (img1.shape[1], img1.shape[0]))
img1.shape, img2.shape
```

Python

```
((1000, 1500, 3), (1000, 1500, 3))
```

합치기~

```
blended = cv2.addWeighted(src1=img1, alpha=0.5, src2=img2, beta=0.5, gamma=0)  
plt.imshow(blended)
```

Python



다시 **resize**

```
img2 = cv2.resize(img2, (600, 600))  
plt.imshow(img2)
```

Python



이건 offset 작업

```
large_img = img1  
small_img = img2  
  
x_offset = 0  
y_offset = 0
```

Python

픽셀 교체~

```
x_end = x_offset + small_img.shape[1]  
y_end = y_offset + small_img.shape[0]  
  
large_img[y_offset:y_end, x_offset:x_end] = small_img
```

Python

흠...

```
plt.imshow(large_img)
```

Python



워터마크

다시 시작해봅시다

```
img1 = cv2.imread("./data/sample.jpg")
img1 = cv2.cvtColor(img1, cv2.COLOR_BGR2RGB)
img2 = cv2.imread("./data/watermark_no_copy.png")
img2 = cv2.cvtColor(img2, cv2.COLOR_BGR2RGB)
img2 = cv2.resize(img2, (600, 600))

img1.shape, img2.shape
```

Python

```
((1000, 1500, 3), (600, 600, 3))
```

offset을 계산하고

```
x_offset = img1.shape[1] - img2.shape[1]
y_offset = img1.shape[0] - img2.shape[0]
x_offset, y_offset
```

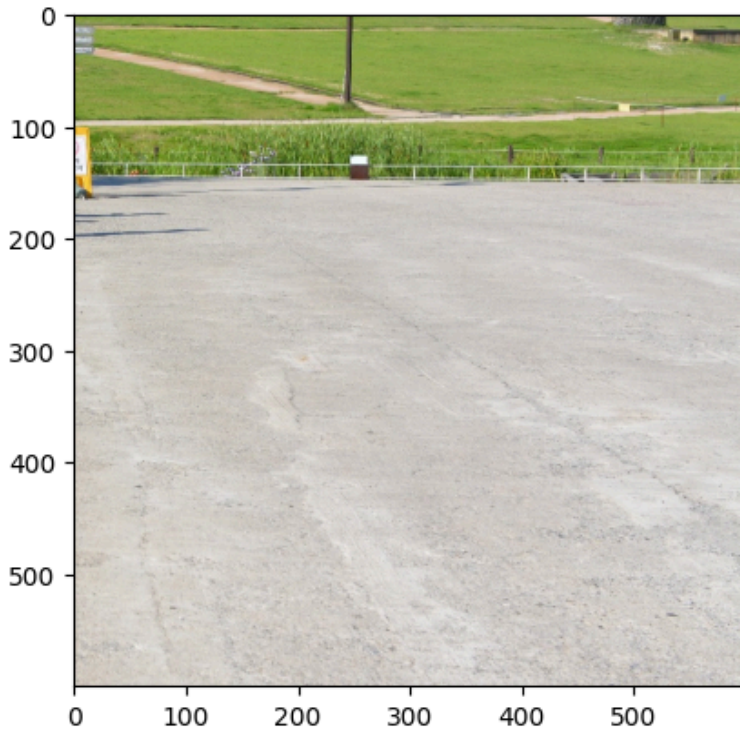
Python

```
(900, 400)
```

offset에 의해 잘린 영역만 본다면~

```
roi = img1[y_offset : img1.shape[0], x_offset : img1.shape[1]]  
plt.imshow(roi)
```

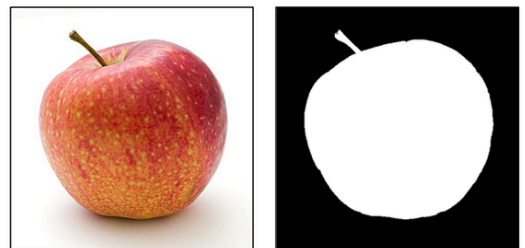
Python



잠깐. roi란?

▪ ROI

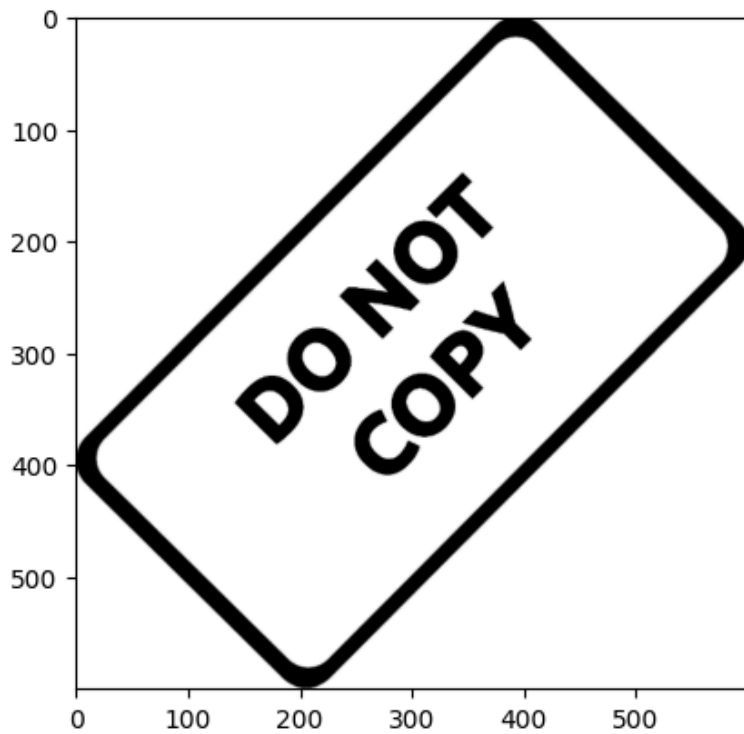
- Region of Interest, 관심 영역
- 영상에서 특정 연산을 수행하고자 하는 임의의 부분 영역



일단 img2를 gray로

```
img2_gray = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)  
plt.imshow(img2_gray, cmap="gray")
```

Python

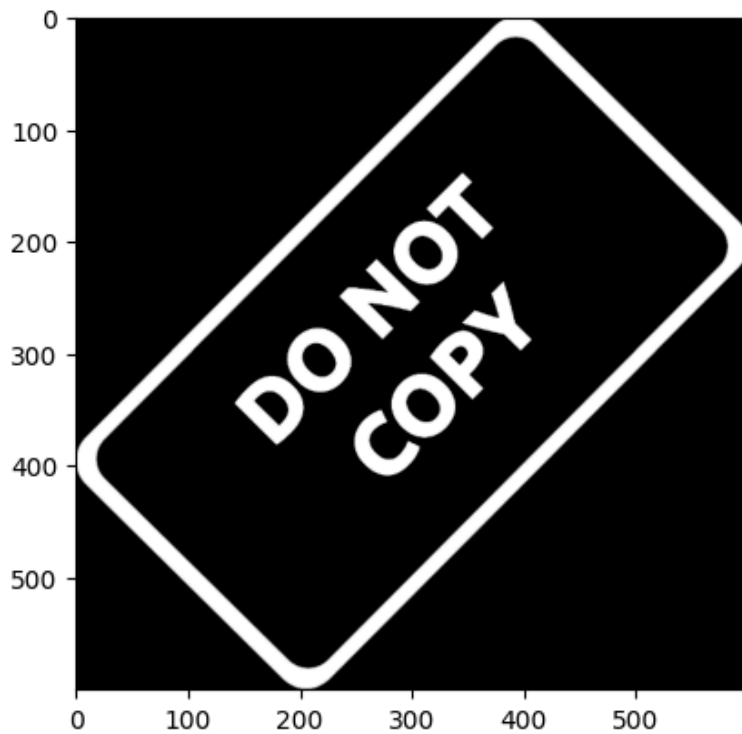


gray에서는 255가 흰색, 0이 검은색~

이 관계를 뒤집는다

```
mask_inv = cv2.bitwise_not(img2_gray)
plt.imshow(mask_inv, cmap="gray")
```

Python



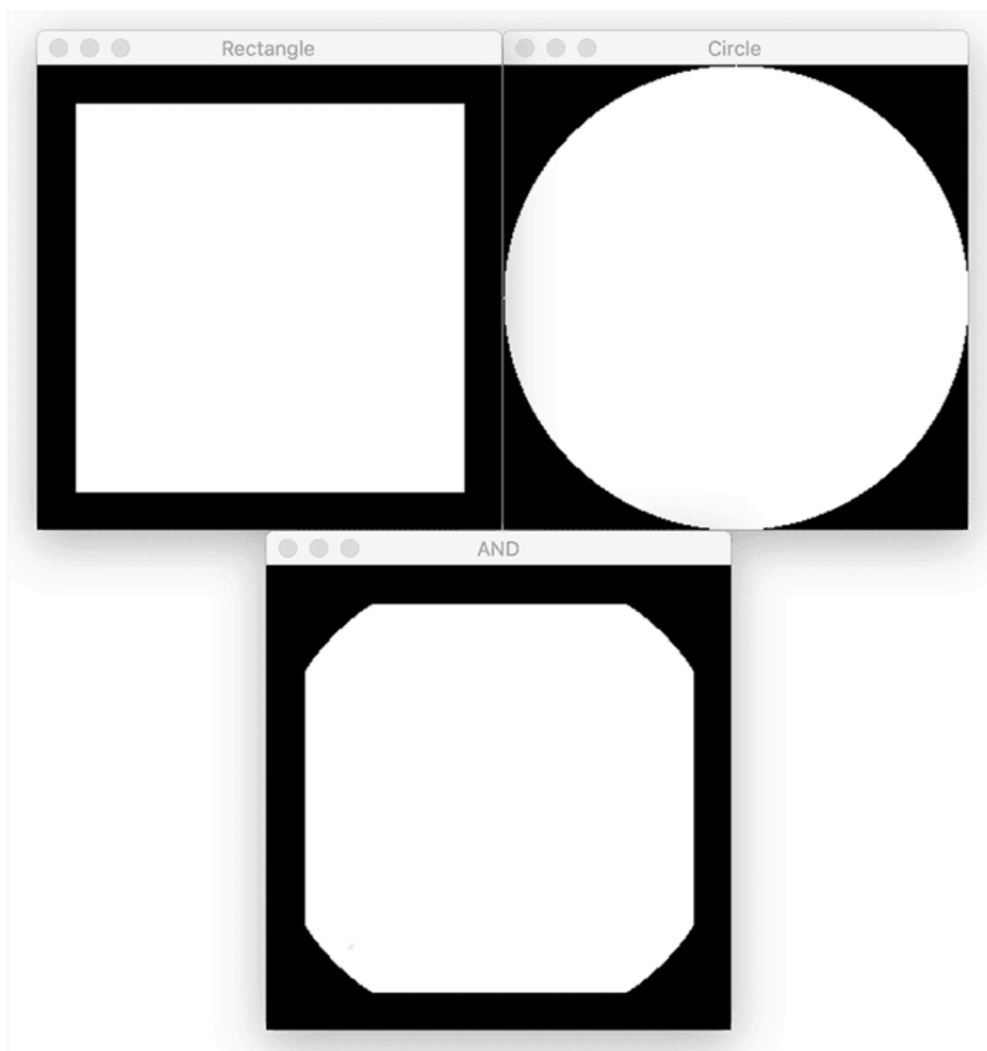
gray니까 shape은 2차원

```
mask_inv.shape
```

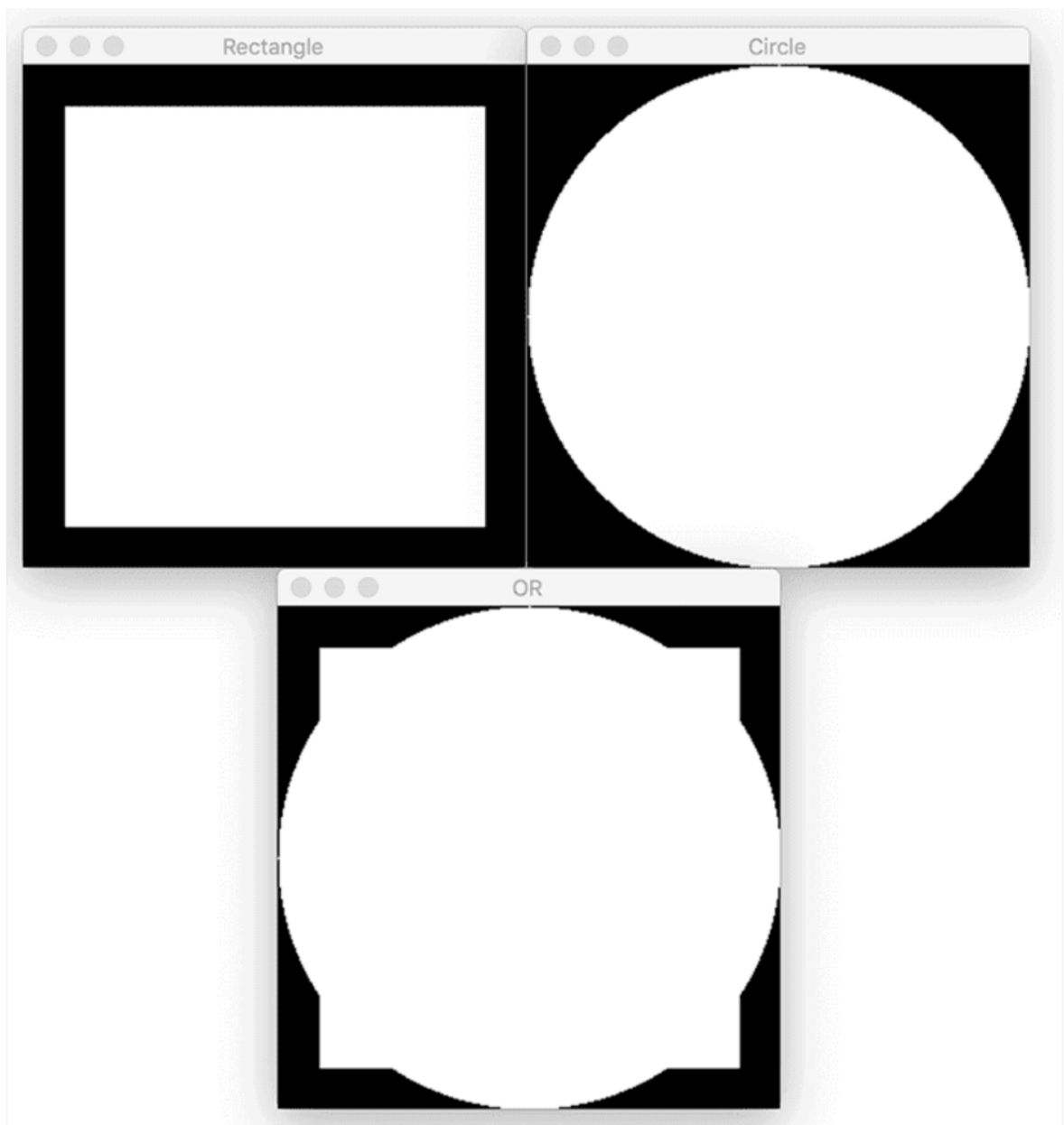
Python

(600, 600)

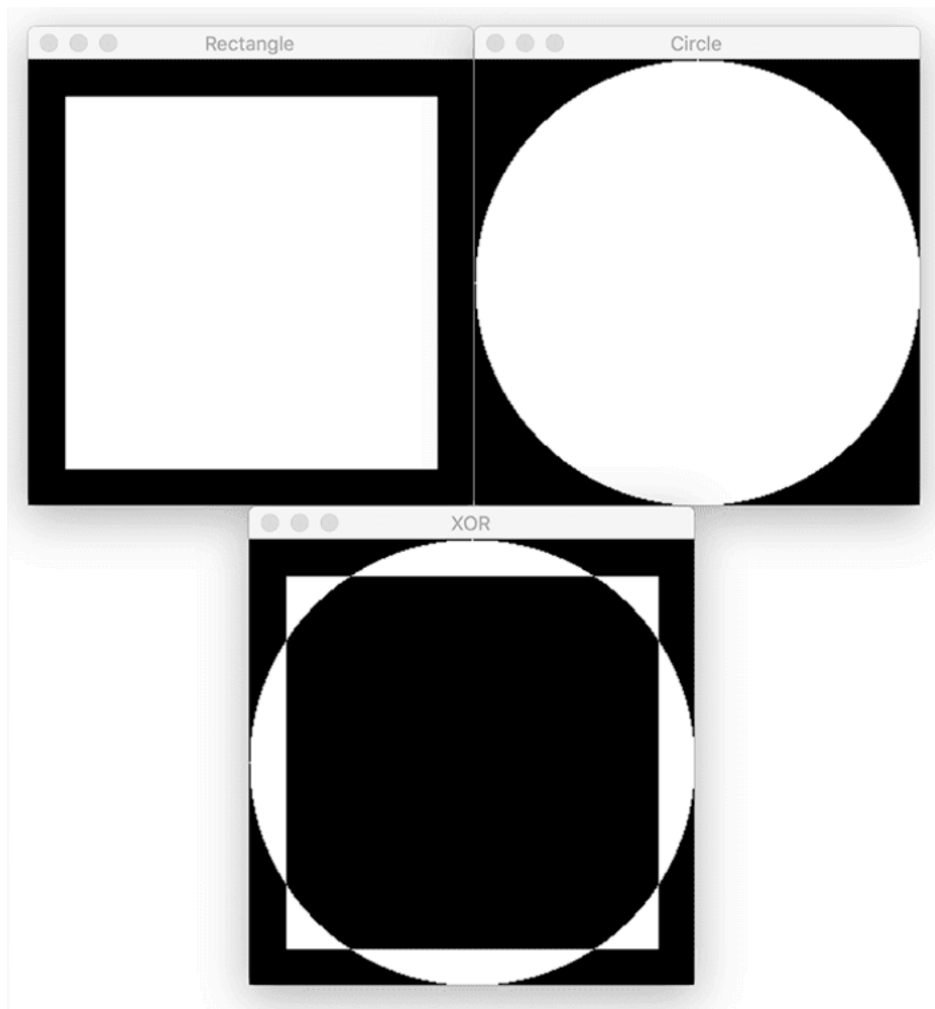
잠깐 bitwise_and



bitwise_or



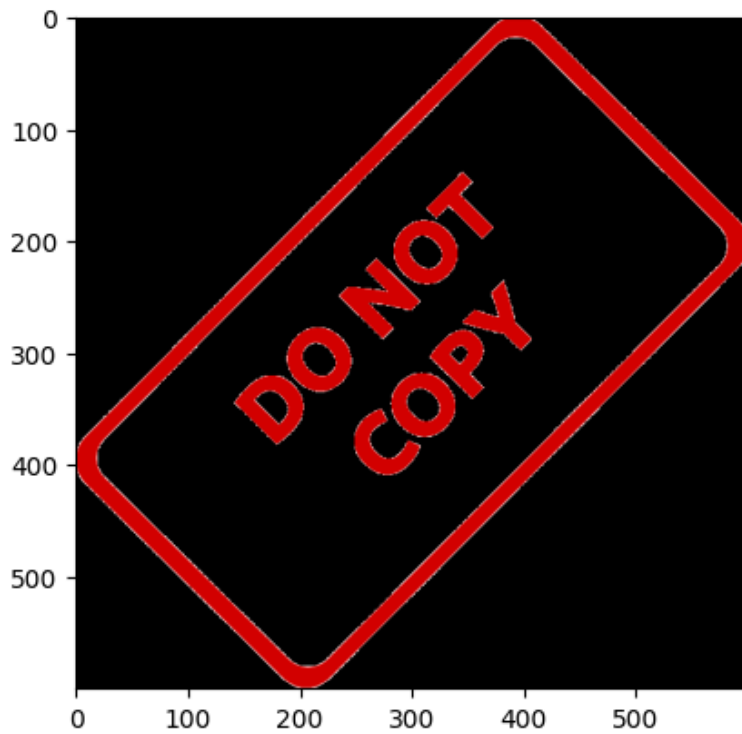
bitwise_xor



까망색배경, 즉, 0으로 배경이 채워지게~

```
fg = cv2.bitwise_or(img2, img2, mask=mask_inv)  
plt.imshow(fg)
```

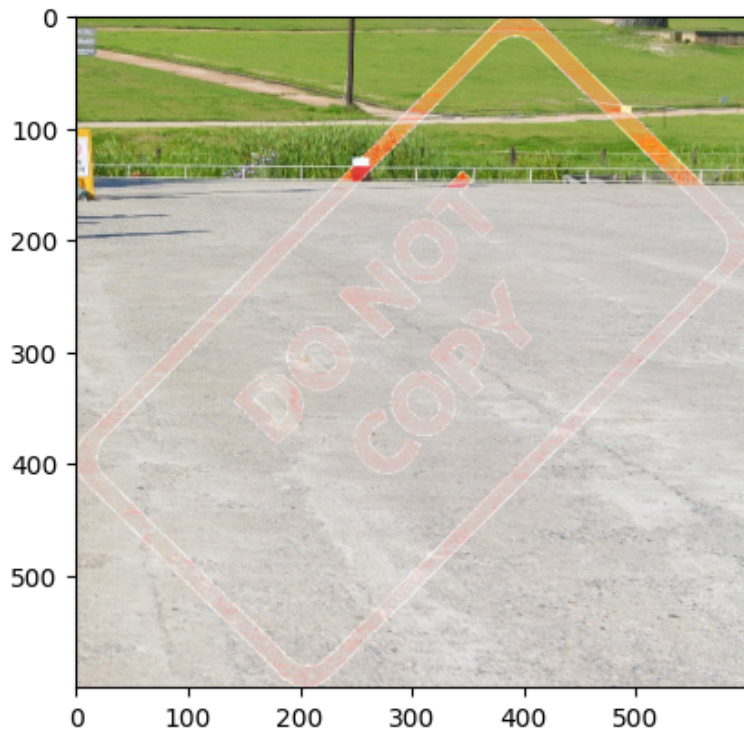
Python



roi에다가 덮어버림

```
final_roi = cv2.bitwise_or(roi, fg)  
plt.imshow(final_roi)
```

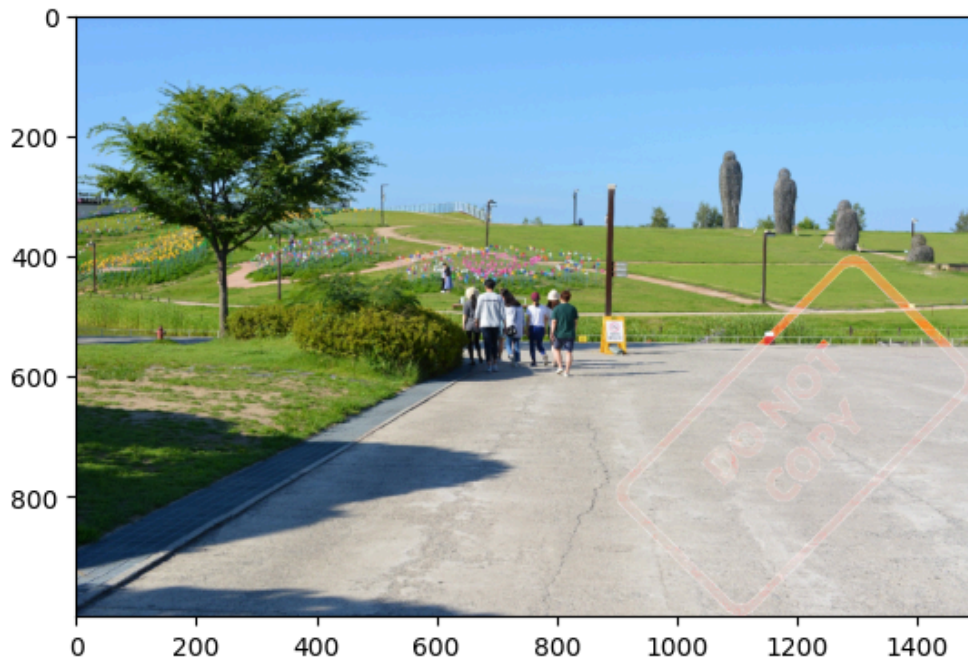
Python



짤!

```
img1[y_offset : y_offset + img2.shape[0], x_offset : x_offset + img2.shape[1]] = (  
    | final_roi  
    )  
plt.imshow(img1)
```

Python



mask 연습

이런 소스에

```
src = cv2.imread("./data/airplane.bmp")  
src = cv2.cvtColor(src, cv2.COLOR_BGR2RGB)  
plt.imshow(src)
```

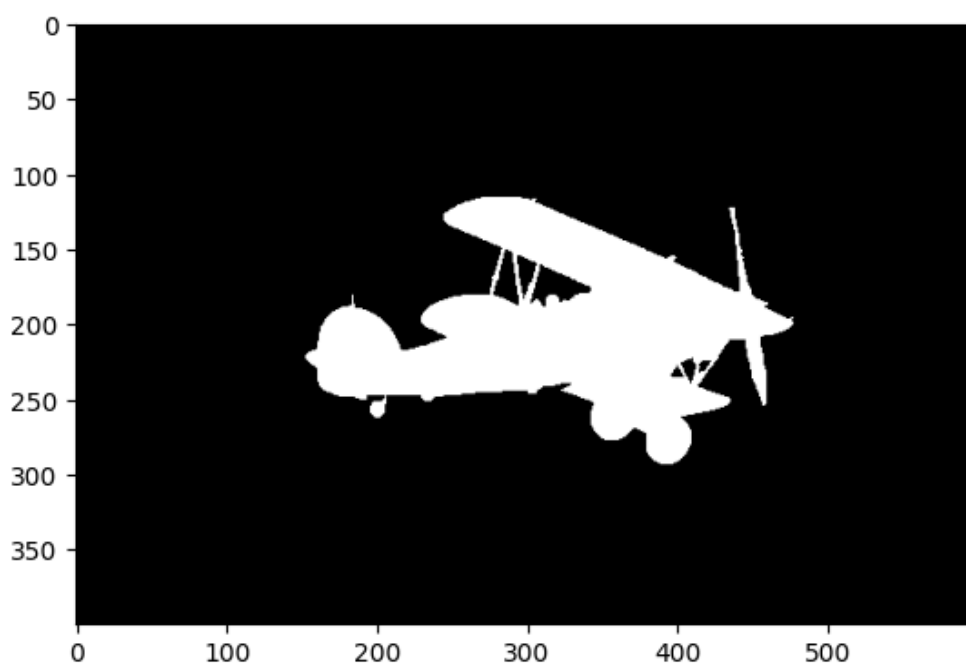
Python



mask가 될 그림을 가져오고

```
mask = cv2.imread("./data/mask_plane.bmp", cv2.IMREAD_GRAYSCALE)  
plt.imshow(mask, cmap="gray")
```

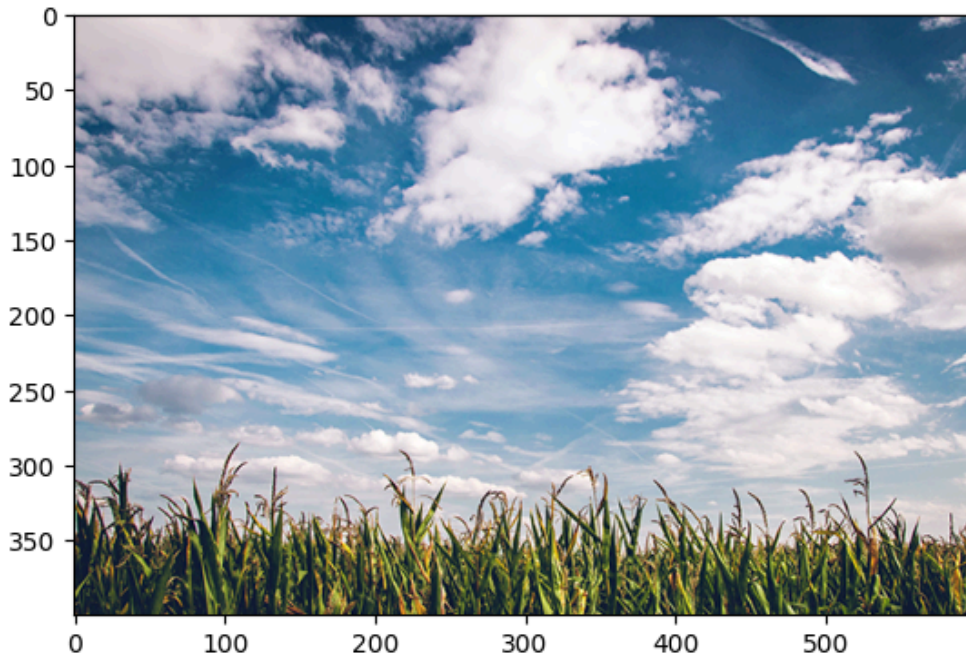
Python



이 배경에 넣고 싶다면~

```
dst = cv2.imread("./data/field.bmp")
dst = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB)
plt.imshow(dst)
```

Python

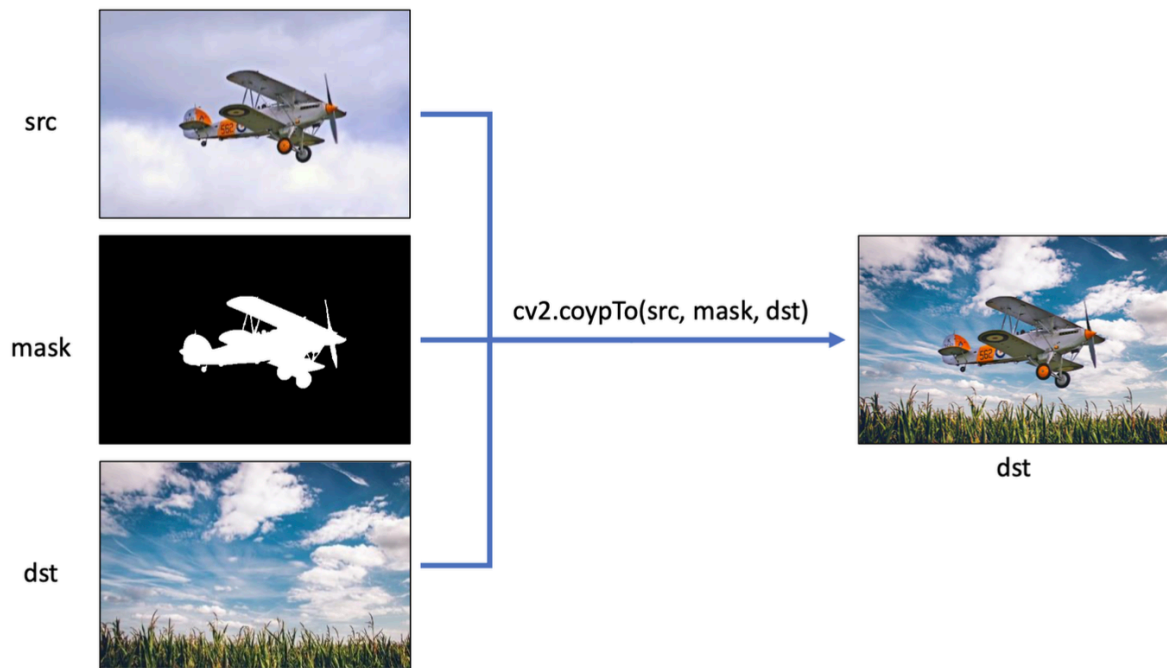


이번에는 copyTo

```
cv2.copyTo(src, mask, dst=None) -> dst
```

- src: 입력 영상
- mask: 마스크 영상. cv2.CV_8U. (numpy.uint8)
0이 아닌 픽셀에 대해서만 복사 연산을 수행.
- dst: 출력 영상. 만약 src와 크기 및 타입이 같은 dst를 입력으로 지정하면 dst를 새로 생성하지 않고 연산을 수행.
그렇지 않으면 dst를 새로 생성하여 연산을 수행한 후 반환함.

이렇게 하고 싶다는 거지



황당할 만큼 간단함

```
plt.imshow(cv2.cvtColor(src, mask, dst));
```

✓ 0.1s

Python





PinkLAB YouTube Channel

PinkLAB Studio — Robotics, AI, ROS2 and more!

 **Subscribe on YouTube**

https://www.youtube.com/@pinklab_studio